МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники

ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине

«Проектирование вычислительных систем»

Вариант: 5

Работу выполнили:
Студенты группы P3411
Болорболд Аригуун
Кравцов Кирилл Денисович

Преподаватель:
Пинкевич Василий Юрьевич

г. Санкт-Петербург
2025

Содержание

1. Текст задания 3
2. Выполнение 4
   2.1. Распиновка 4
   2.2. Конфигурация частоты ядра 4
   2.3. Блок-схема: 5
   2.4. Описание работы алгоритма 5
   2.5. Код драйвера 5
3. Вывод 15

1. Текст задания

Разработать программу, которая использует интерфейс I2C для считывания нажатий кнопок клавиатуры стенда SDK-1.1.

Подсистема опроса клавиатуры должна удовлетворять следующим требованиям:

- реализуется защита от дребезга;
- нажатие кнопки фиксируется сразу после того, как было обнаружено, что кнопка нажата (с учетом защиты от дребезга), а не в момент отпускания кнопки; если необходимо, долгое нажатие может фиксироваться отдельно;
- кнопка, которая удерживается дольше, чем один цикл опроса, не считается повторно нажатой до тех пор, пока не будет отпущена (нет переповторов);
- распознается и корректно обрабатывается множественное нажатие (при нажатии более чем одной кнопки считается, что ни одна кнопка не нажата, если это не противоречит требованиям к программе);
- всем кнопкам назначаются коды от 1 до 12 (порядок на усмотрение исполнителей).

Программа должна иметь два режима работы, переключение между которыми производится по нажатию кнопки на боковой панели стенда:

- режим тестирования клавиатуры;
- прикладной режим.

Уведомление о смене режима выводится в UART.

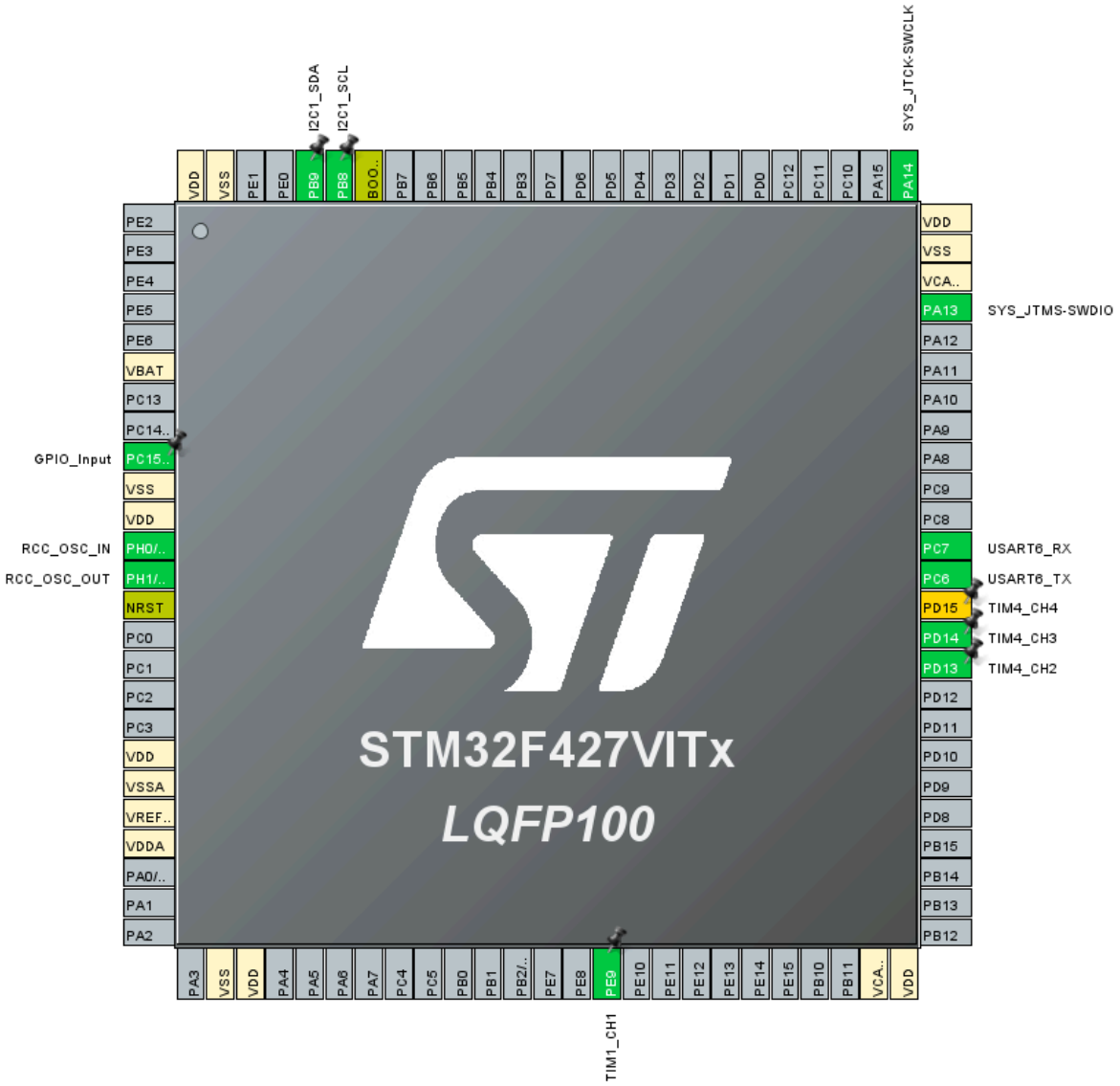
В режиме тестирования клавиатуры программа выводит в UART коды нажатых кнопок.

В прикладном режиме программа обрабатывает нажатия кнопок и выполняет действия в соответствии с вариантом задания.

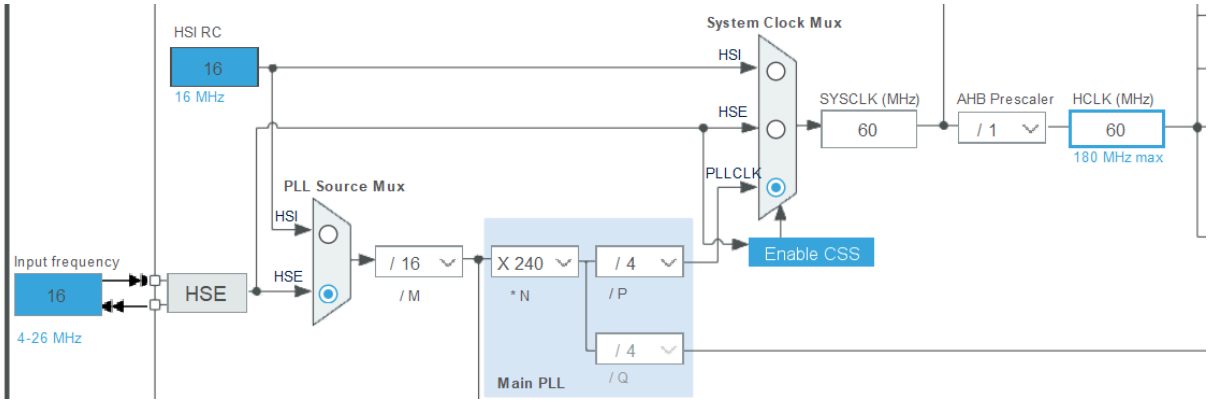
Задания аналогичны вариантам лабораторной работы №3, за исключением того, что ввод символов должен выполняться не с клавиатуры через UART, а с помощью клавиатуры стенда. Выбор кнопок клавиатуры стенда, играющих роль кнопок клавиатуры компьютера должен выполняться по усмотрению исполнителей. В отчете необходимо привести описание функций кнопок в реализованной программе.

2. Выполнение

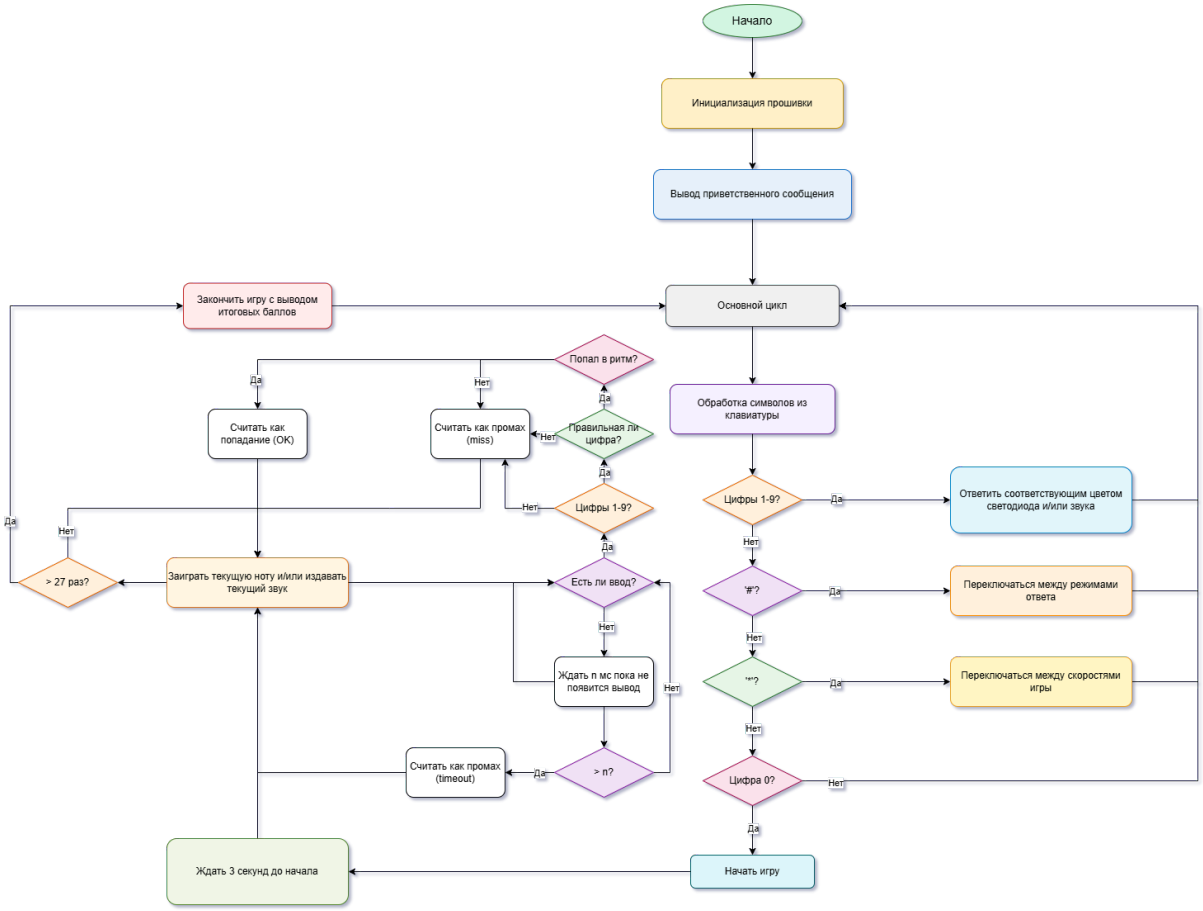
2.1. Распиновка



2.2. Конфигурация частоты ядра



2.3. Блок-схема:



2.4. Описание работы алгоритма

Появится сообщение на экране, после чего можно будет тестировать код через клавиатуру стенда. Если встретится цифр от 1 до 9, то стенд будет мигать соответствующим цветом и издавать звук. Если встретится символ «а» (кнопка №12), то изменится режим ответа на цифр — либо только светодиод, либо только звук, либо оба. Если пользователь (дальше «игрок») нажмет на «0», то начнется последовательность, где «игроку» надо будет в точь нажать на цифры, иначе это считается как промах, а если вовсе не нажмет, то после определенной времени будет указываться таймаут. В итоге программа считает, сколько раз игрок попал в ритм. При ввода символа «+» (кнопка №10) переключается скорость игры: 1с, 600мс, 300мс.

2.5. Код драйвера

```
#define NUM_PULSES 9
#define SEQUENCE_LENGTH 27

#define PCA9538_ADDR 0xE2
#define PCA9538_INPUT_REG 0x00
#define PCA9538_OUTPUT_REG 0x01
#define PCA9538_CONFIG_REG 0x03
```

```
#define KEYBOARD_ROWS 4
#define KEYBOARD_COLS 3
#define DEBOUNCE_TIME 50

const uint16_t FREQUENCIES[NUM_PULSES] = {
    262, 294, 330, 349, 392, 440, 494, 523, 587
};

typedef struct {
    uint8_t green;
    uint8_t yellow;
    uint8_t red;
    uint16_t freq;
    char symbol;
} Pulse_t;

const Pulse_t PULSES[NUM_PULSES] = {
    {20,  0,   0,  262, '1'},
    {50,  0,   0,  294, '2'},
    {100, 0,   0,  330, '3'},
    {0,   20,  0,  349, '4'},
    {0,   50,  0,  392, '5'},
    {0,   100, 0,  440, '6'},
    {0,   0,   20, 494, '7'},
    {0,   0,   50, 523, '8'},
    {0,   0,   100, 587, '9'}
};

typedef enum {
    SPEED_SLOW = 0,
    SPEED_MEDIUM = 1,
    SPEED_FAST = 2
} Speed_t;

const uint16_t SPEED_DURATIONS[3] = {1000, 600, 300};

typedef enum {
    MODE_BOTH = 0,
    MODE_LED_ONLY = 1,
    MODE_SOUND_ONLY = 2
} PlayMode_t;

typedef enum {
    GAME_IDLE = 0,
    GAME_COUNTDOWN = 1,
```

```
    GAME_PLAYING = 2,
    GAME_FINISHED = 3
} GameState_t;

extern TIM_HandleTypeDef htim1;
extern TIM_HandleTypeDef htim4;
extern TIM_HandleTypeDef htim6;
extern UART_HandleTypeDef huart6;
extern I2C_HandleTypeDef hi2c1;

static Speed_t current_speed = SPEED_SLOW;
static PlayMode_t play_mode = MODE_BOTH;
static GameState_t game_state = GAME_IDLE;

static uint8_t sequence[SEQUENCE_LENGTH];
static uint8_t sequence_pos = 0;
static uint32_t score = 0;
static uint8_t mistakes = 0;
static uint8_t input_received = 0;
static char trace[SEQUENCE_LENGTH];

static uint32_t countdown_start = 0;

static uint8_t keyboard_state[KEYBOARD_ROWS][KEYBOARD_COLS];
static uint32_t keyboard_debounce[KEYBOARD_ROWS][KEYBOARD_COLS];
static uint8_t keyboard_pressed[KEYBOARD_ROWS][KEYBOARD_COLS];

const char KEYBOARD_MAP[KEYBOARD_ROWS][KEYBOARD_COLS] = {
    {'1', '2', '3'},
    {'4', '5', '6'},
    {'7', '8', '9'},
    {'+', '0', 'a'}
};

void SystemClock_Config(void);
static void SetPWM(uint8_t green, uint8_t yellow, uint8_t red);
static void SetBuzzer(uint16_t freq);
static void StopBuzzer(void);
static void PlayPulse(uint8_t pulse_idx);
static void StopPulse(void);
static void GenerateSequence(void);
static void StartGame(void);
static void StopGame(void);
static void ProcessInput(char c);
static void PrintGameStats(void);
static void UART_SendString(const char* str);
```

```c
static HAL_StatusTypeDef PCA9538_WriteReg(uint8_t reg, uint8_t
data);
static HAL_StatusTypeDef PCA9538_ReadReg(uint8_t reg, uint8_t
*data);
static void Keyboard_Init(void);
static void Keyboard_Scan(void);
static char Keyboard_GetKey(void);

static HAL_StatusTypeDef PCA9538_WriteReg(uint8_t reg, uint8_t
data) {
    return HAL_I2C_Mem_Write(&hi2c1, PCA9538_ADDR, reg, 1, &data,
1, 100);
}

static HAL_StatusTypeDef PCA9538_ReadReg(uint8_t reg, uint8_t
*data) {
    return HAL_I2C_Mem_Read(&hi2c1, PCA9538_ADDR, reg, 1, data, 1,
100);
}

static void Keyboard_Init(void) {
    PCA9538_WriteReg(PCA9538_CONFIG_REG, 0xF0);

    PCA9538_WriteReg(PCA9538_OUTPUT_REG, 0x0F);

    memset(keyboard_state, 0, sizeof(keyboard_state));
    memset(keyboard_debounce, 0, sizeof(keyboard_debounce));
    memset(keyboard_pressed, 0, sizeof(keyboard_pressed));

    UART_SendString("Keyboard initialized\r\n");
}

static void Keyboard_Scan(void) {
    uint32_t now = HAL_GetTick();

    for (uint8_t row = 0; row < KEYBOARD_ROWS; row++) {
        uint8_t output = 0x0F & ~(1 << row);
        PCA9538_WriteReg(PCA9538_OUTPUT_REG, output);

        HAL_Delay(1);

        uint8_t input = 0;
        if (PCA9538_ReadReg(PCA9538_INPUT_REG, &input) == HAL_OK) {
            for (uint8_t col = 0; col < KEYBOARD_COLS; col++) {
                uint8_t bit = 4 + col;
```

```
                uint8_t pressed = !(input & (1 << bit));

                if (pressed != keyboard_state[row][col]) {
                    keyboard_state[row][col] = pressed;
                    keyboard_debounce[row][col] = now;
                } else {
                    if (now - keyboard_debounce[row][col] >=
DEBOUNCE_TIME) {
                        if (pressed && !keyboard_pressed[row][col])
{
                            keyboard_pressed[row][col] = 1;
                        } else if (!pressed &&
keyboard_pressed[row][col]) {
                            keyboard_pressed[row][col] = 0;
                        }
                    }
                }
            }
        }
    }

    PCA9538_WriteReg(PCA9538_OUTPUT_REG, 0x0F);
}

static char Keyboard_GetKey(void) {
    uint8_t pressed_count = 0;
    uint8_t last_row = 0, last_col = 0;

    for (uint8_t row = 0; row < KEYBOARD_ROWS; row++) {
        for (uint8_t col = 0; col < KEYBOARD_COLS; col++) {
            if (keyboard_pressed[row][col]) {
                pressed_count++;
                last_row = row;
                last_col = col;
            }
        }
    }

    if (pressed_count != 1) {
        return 0;
    }

    static uint8_t key_handled[KEYBOARD_ROWS][KEYBOARD_COLS] = {0};

    if (key_handled[last_row][last_col]) {
        return 0;
```

```
    }

    key_handled[last_row][last_col] = 1;

    for (uint8_t row = 0; row < KEYBOARD_ROWS; row++) {
        for (uint8_t col = 0; col < KEYBOARD_COLS; col++) {
            if (!keyboard_pressed[row][col]) {
                key_handled[row][col] = 0;
            }
        }
    }

    return KEYBOARD_MAP[last_row][last_col];
}

static void SetPWM(uint8_t green, uint8_t yellow, uint8_t red) {
    if (play_mode == MODE_SOUND_ONLY) {
        __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_2, 0);
        __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_3, 0);
        __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_4, 0);
        return;
    }

    __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_2, (green * 2000) /
100);
    __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_3, (yellow * 2000) /
100);
    __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_4, (red * 2000) /
100);
}

static void SetBuzzer(uint16_t freq) {
    if (play_mode == MODE_LED_ONLY || freq == 0) {
        HAL_TIM_PWM_Stop(&htim1, TIM_CHANNEL_1);
        return;
    }

    uint32_t timer_clock = 144000000;
    uint16_t prescaler = (timer_clock / 1000000) - 1;
    uint32_t arr = (1000000 / freq) - 1;

    if (arr > 65535) arr = 65535;
    if (arr < 10) arr = 10;

    HAL_TIM_PWM_Stop(&htim1, TIM_CHANNEL_1);
```

```
    htim1.Instance->PSC = prescaler;
    htim1.Instance->ARR = arr;
    htim1.Instance->CCR1 = arr / 2;
    htim1.Instance->CNT = 0;
    htim1.Instance->EGR = TIM_EGR_UG;

    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
}

static void StopBuzzer(void) {
    HAL_TIM_PWM_Stop(&htim1, TIM_CHANNEL_1);
}

static void PlayPulse(uint8_t pulse_idx) {
    const Pulse_t* p = &PULSES[pulse_idx];
    SetPWM(p->green, p->yellow, p->red);
    SetBuzzer(p->freq);
}

static void StopPulse(void) {
    SetPWM(0, 0, 0);
    StopBuzzer();
}

static void GenerateSequence(void) {
    for (int i = 0; i < SEQUENCE_LENGTH; i++) {
        sequence[i] = i % NUM_PULSES;
    }
}

static void StartGame(void) {
    game_state = GAME_COUNTDOWN;
    countdown_start = HAL_GetTick();
    score = 0;
    mistakes = 0;
    sequence_pos = 0;
    memset(trace, 0, sizeof(trace));
    GenerateSequence();

    UART_SendString("\r\n=== GAME STARTING IN 3 SECONDS ===\r\n");
}

static void StopGame(void) {
    StopPulse();
    HAL_TIM_Base_Stop_IT(&htim6);
    game_state = GAME_FINISHED;
```

```
    PrintGameStats();
}

static void PrintGameStats(void) {
    char buf[256];
    snprintf(buf, sizeof(buf),
             "\r\n=== GAME FINISHED ===\r\n"
             "Score: %lu\r\n"
             "Correct: %d / %d\r\n"
             "Trace: ",
             score, SEQUENCE_LENGTH - mistakes, SEQUENCE_LENGTH);
    UART_SendString(buf);

    for (int i = 0; i < SEQUENCE_LENGTH; i++) {
        if (trace[i] == '+') {
            UART_SendString("[OK]");
        } else if (trace[i] == '-') {
            UART_SendString("[MISS]");
        } else {
            UART_SendString("[TIMEOUT]");
        }
    }
    UART_SendString("\r\n");
    game_state = GAME_IDLE;
}

static void ProcessInput(char c) {
    if (c == 0) return;

    char buf[128];

    if (c == '0') {
        if (game_state == GAME_IDLE) {
            StartGame();
        } else if (game_state == GAME_COUNTDOWN || game_state ==
GAME_PLAYING) {
            UART_SendString("\r\n=== GAME STOPPED ===\r\n");
            StopGame();
        }
        return;
    }

    if (game_state == GAME_IDLE) {
        for (int i = 0; i < NUM_PULSES; i++) {
            if (c == PULSES[i].symbol) {
                snprintf(buf, sizeof(buf), "\r\nKey pressed:
```

```
'%c'\r\n", c);
                UART_SendString(buf);

                PlayPulse(i);
                HAL_Delay(200);
                StopPulse();

                snprintf(buf, sizeof(buf),
                         "Pulse '%c': Color=%s, Brightness=%d%%,
Frequency=%d Hz\r\n",
                         c,
                         PULSES[i].green ? "Green" :
(PULSES[i].yellow ? "Yellow" : "Red"),
                         PULSES[i].green + PULSES[i].yellow +
PULSES[i].red,
                         PULSES[i].freq);
                UART_SendString(buf);
                return;
            }
        }

        if (c == '+') {
            current_speed = (current_speed + 1) % 3;
            const char* speed_names[] = {"SLOW", "MEDIUM", "FAST"};
            snprintf(buf, sizeof(buf), "Speed: %s (%d ms)\r\n",
                     speed_names[current_speed],
                     SPEED_DURATIONS[current_speed]);
            UART_SendString(buf);
        }
        else if (c == 'a' || c == 'A') {
            play_mode = (play_mode + 1) % 3;
            const char* mode_names[] = {"LED + SOUND", "LED ONLY",
"SOUND ONLY"};
            snprintf(buf, sizeof(buf), "Play mode: %s\r\n",
mode_names[play_mode]);
            UART_SendString(buf);
        }
    }
    else if (game_state == GAME_PLAYING) {
        if (input_received) return;

        uint8_t expected = sequence[sequence_pos];
        if (c == PULSES[expected].symbol) {
            trace[sequence_pos] = '+';
            score += 10 * (current_speed + 1);
        } else {
```

```
            trace[sequence_pos] = '-';
            mistakes++;
        }
        input_received = 1;
    }
}

static void UART_SendString(const char* str) {
    HAL_UART_Transmit(&huart6, (uint8_t*)str, strlen(str),
HAL_MAX_DELAY);
}

int main(void)
{
  HAL_Init();
  SystemClock_Config();

  /* USER CODE BEGIN 2 */
  MX_GPIO_Init();
  MX_USART6_UART_Init();
  MX_I2C1_Init();
  MX_TIM6_Init();
  MX_TIM4_Init();
  MX_TIM1_Init();

  UART_SendString("\r\n\r\n=== RHYTHM GAME WITH I2C KEYBOARD
===\r\n");
  UART_SendString("Lab 4: I2C Interface\r\n\r\n");

  HAL_Delay(100);
  Keyboard_Init();

  HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_2);
  HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_3);
  HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_4);
  SetPWM(0, 0, 0);

  UART_SendString("Commands:\r\n");
  UART_SendString("  1-9 : Test pulses\r\n");
  UART_SendString("  *   : Change speed\r\n");
  UART_SendString("  #   : Change play mode\r\n");
  UART_SendString("  0   : Start/Stop game\r\n");
  UART_SendString("\r\nReady!\r\n");
  uint32_t last_scan = 0;

  while (1)
```

```
  {
    uint32_t now = HAL_GetTick();

    if (now - last_scan >= 10) {
        last_scan = now;
        Keyboard_Scan();

        char key = Keyboard_GetKey();
        if (key != 0) {
            ProcessInput(key);
        }
    }

    if (game_state == GAME_COUNTDOWN) {
        if (now - countdown_start >= 3000) {
            game_state = GAME_PLAYING;
            sequence_pos = 0;
            input_received = 0;
            PlayPulse(sequence[0]);

            __HAL_TIM_SET_COUNTER(&htim6, 0);
            __HAL_TIM_SET_AUTORELOAD(&htim6,
SPEED_DURATIONS[current_speed] - 1);
            HAL_TIM_Base_Start_IT(&htim6);

            UART_SendString("=== GO! ===\r\n");
        }
    }
  }
}
```

3. Вывод

В ходе выполнения работы мы ознакомились с принципом работы I2C и матричной клавиатуры, откуда нам требовалось обработать события нажатия кнопки для правильной работы программы, логика которого сохранилась с прошлой работы.